

Reinforcement Learning



1. Introduction to Reinforcement Learning
2. Markov Decision Processes (MDPs)
3. Policy and Value Functions
- 4. Dynamic Programming (DP) for RL**
5. Model-Free methods

- Let's write a function that returns the n-th Fibonacci number.

- Fibonacci's general rule:

- $\text{Fib}(0) = 0$
- $\text{Fib}(1) = 1$
- $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

$\text{Fib}(0) = 0$

$\text{Fib}(1) = 1$

$\text{Fib}(2) = 1$

$\text{Fib}(3) = 2$

$\text{Fib}(4) = 3$

$\text{Fib}(5) = 5$

$\text{Fib}(6) = 8$

...

Naive approach

```
def fib(n):  
    if n <= 1:  
        result = 1  
  
    else:  
        result = fib(n - 1) + fib(n - 2)  
  
    return result
```

```
print(fib(7))    Output: 13
```

```
print(fib(13))   Output: TIMEOUT
```

- **Memoization** = caching the results of recursive subproblems to avoid redundant computation
- A technique where you store the results of recursive calls and reuse them instead of recomputing them.

```
memo = {}  
  
def fib(n):  
    if n in memo:  
        return memo[n]  
  
    if n <= 1:  
        result = 1  
    else:  
        result = fib(n - 1) + fib(n - 2)  
  
    memo[n] = result  
    return result
```

```
print(fib(7))    Output: 13
```

```
print(fib(13))   Output: 12586269025
```

Computing optimal policies through Recursion + Memoization.

The Premise:

DP requires a perfect model of the environment (a finite Markov Decision Process).

The Mechanism:

DP relies on turning recursive Bellman equations into iterative update rules.

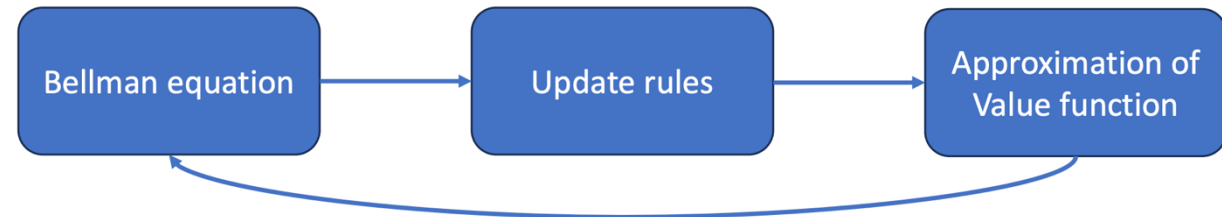
The Two Pillars:

Recursion:

Breaking the problem into subproblems (evaluating one state based on the next).

Memoization:

Storing and reusing the computed values (value functions) to avoid redundant computation.

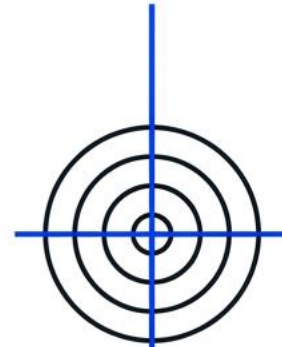


- Dozens of distinct algorithms exist beneath the math.
- They all answer just two independent questions.

Question 1:

- What is the Goal ?

Question 1: The Goal

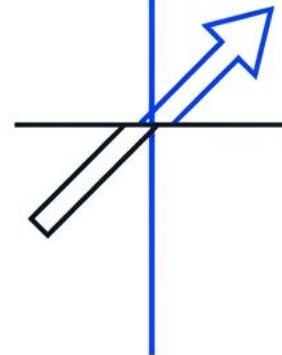


Prediction

Evaluate a given policy.

Input: Current rules → Output: Expected value

(No improvement step)



Control

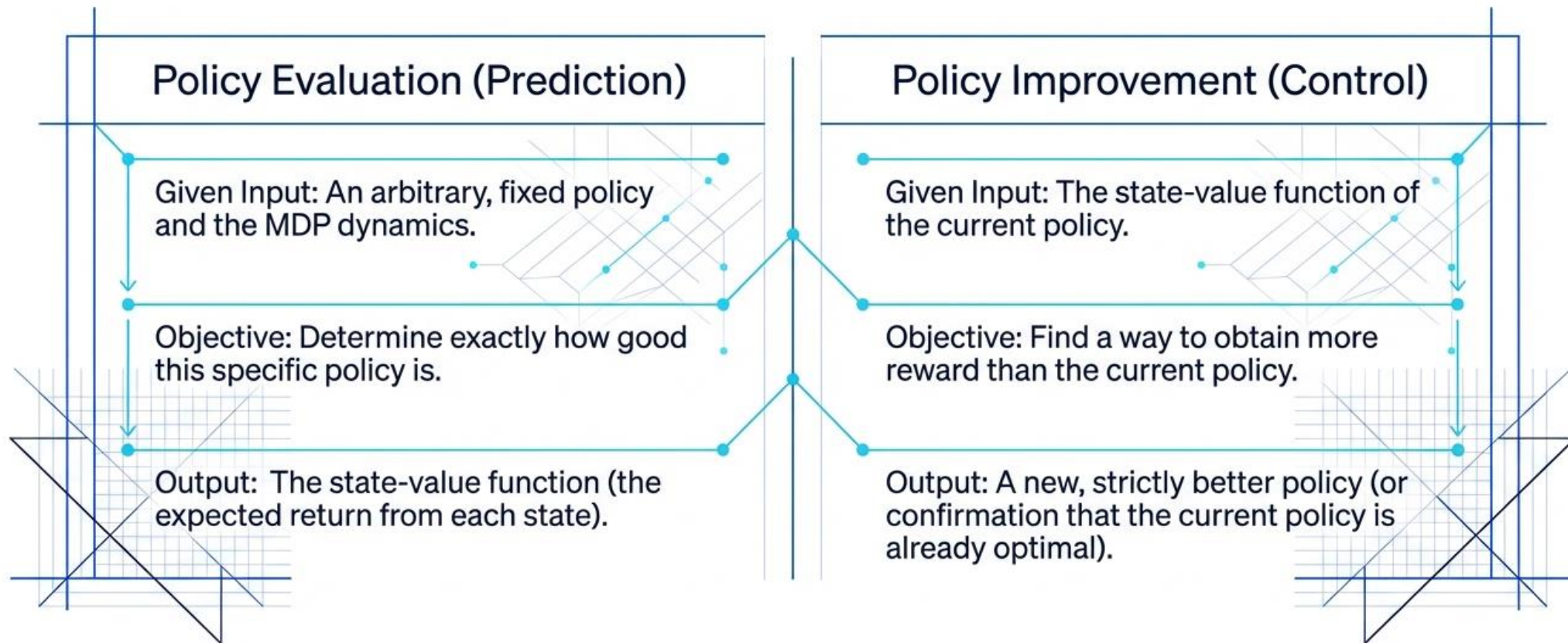
Find the best policy.

Output: Optimal strategy

(Requires improvement step)

The Two Steps of the Dance: Prediction vs. Control

Disentangling the evaluation of a policy from the optimization of behavior.



- The “prediction problem”
- Compute the state value v_π for an arbitrary policy π

$$\begin{aligned}v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] \\&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\&= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\&= \sum_a \pi(a|s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]\end{aligned}$$

If the dynamics
of the
environment is
known the
solution is
straight forward

- where $\pi(a|s)$ is the probability of taking action a in state s under policy π ,
- and the expectations are subscripted by π to indicate that they are conditional on π being followed

- The “prediction problem”
- Compute the state value v_π for an arbitrary policy π

$$\begin{aligned} v_{k+1}(s) &\doteq \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')] \end{aligned}$$

- where $\pi(a|s)$ is the probability of taking action a in state s under policy π ,
- and the expectations are subscripted by π to indicate that they are conditional on π being followed

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

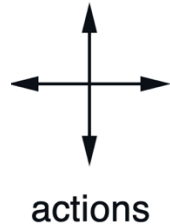
$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Example: Gridworld

Consider the 4×4 gridworld shown below.



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R_t = -1$
on all transitions

v_k for the
random policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

$$0.25 * (-1 + 0) + 0.25 * (-1 + 0) + 0.25 * (-1 + 0) + 0.25 * (-1 + 0) = -1$$

- We know how good it is to follow the current policy from s , that is $v_\pi(s)$, but would it be better or worse to change to the new policy?
- If we select a in s according to π

$$\begin{aligned} q_\pi(s, a) &\doteq \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]. \end{aligned}$$

- Policy improvement theorem:

Let π and π' be any pair of deterministic policies such that, for all $s \in \mathcal{S}$,

$$q_\pi(s, \pi'(s)) \geq v_\pi(s).$$

Then the policy π' must be as good as, or better than, π . That is, it must obtain greater or equal expected return from all states $s \in \mathcal{S}$:

$$v_{\pi'}(s) \geq v_\pi(s).$$

- So far, we have evaluated a change in the policy
- Extend the idea, to all states and all possible actions
- Select a that is best according to $q_\pi(s, a)$
- Consider this the new greedy policy π'

$$\begin{aligned}
 \pi'(s) &\doteq \operatorname{argmax}_a q_\pi(s, a) \\
 &= \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\
 &= \operatorname{argmax}_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')],
 \end{aligned}$$

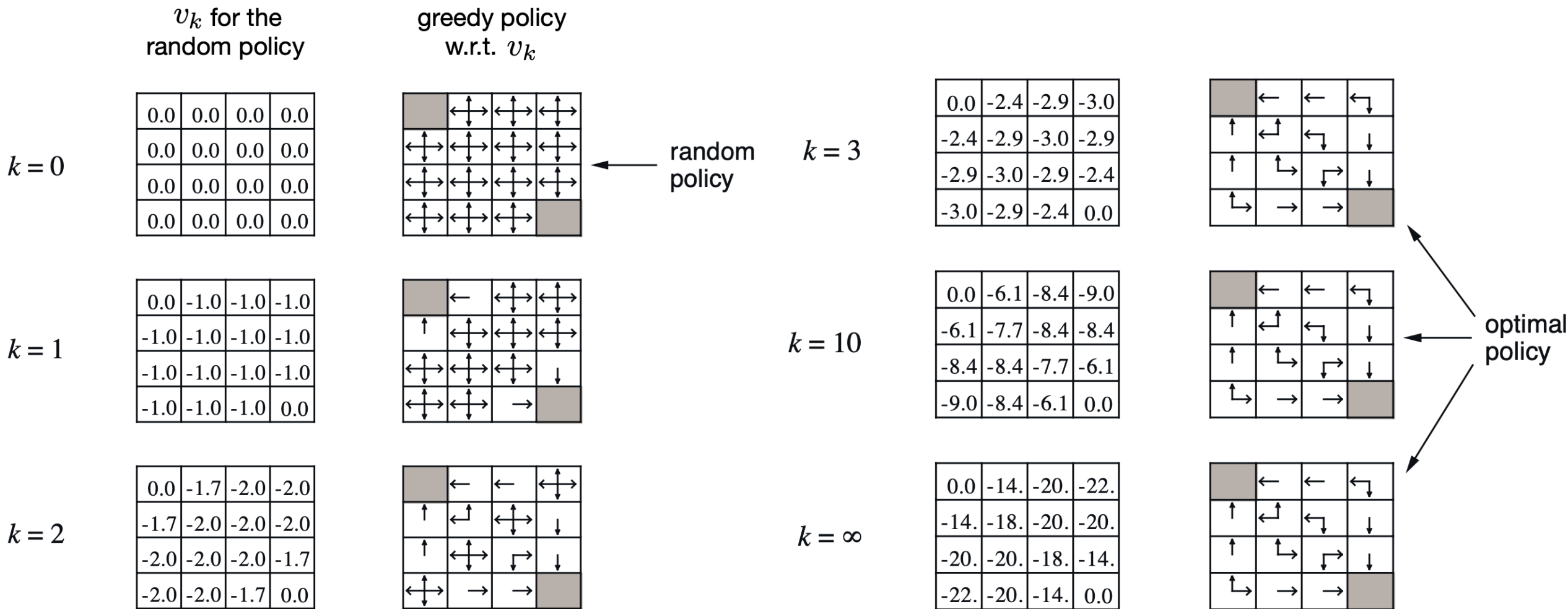
Policy π and $v_\pi(s)$

				0.0	-1.7	-2.0	-2.0
				-1.7	-2.0	-2.0	-2.0
				-2.0	-2.0	-2.0	-1.7
				-2.0	-2.0	-1.7	0.0



New greedy π'



- Sequence of monotonically improving policies and value functions

- E – evaluation
- I – improvement

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} v_*,$$

- Finite MDP has only a finite number of policies, so this process must converge to an optimal policy and optimal value function

Step 1 (Evaluate): Make the <u>value function</u> completely accurate with respect to the current policy.	Step 2 (Improve): Make the policy entirely greedy with respect to the current <u>value function</u>.
The Interaction: Greedyfying ruins the accuracy of the value function; evaluating ruins the greediness of the policy. They must alternate.	Convergence: Because a finite MDP has a finite number of policies, this alternating sequence must eventually lock into an optimal policy and optimal value function.

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

$policy_stable \leftarrow true$

For each $s \in \mathcal{S}$:

$old_action \leftarrow \pi(s)$

$\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If $old_action \neq \pi(s)$, then $policy_stable \leftarrow false$

If $policy_stable$, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Problem:

Oscillation may occur

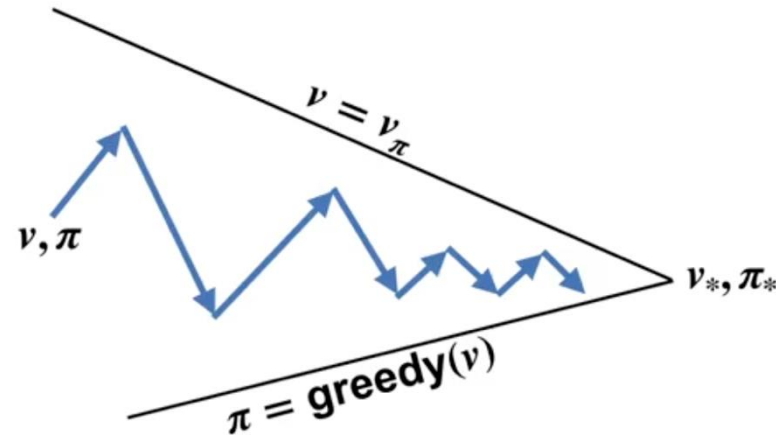
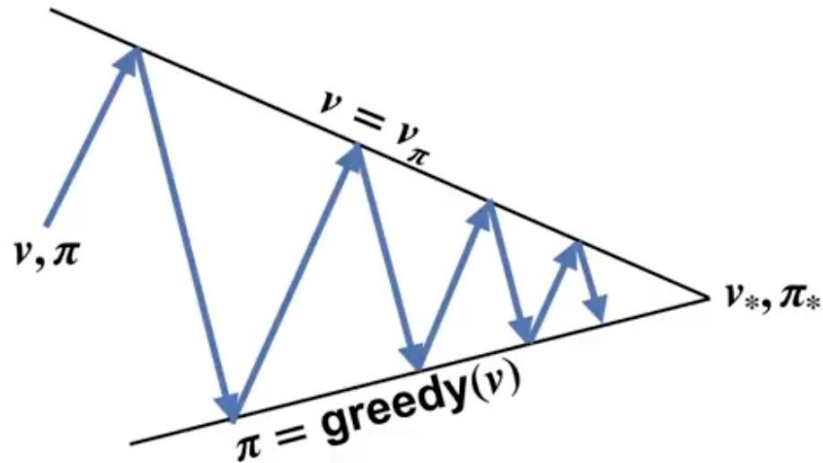
if $v_{\pi'}(s) = v_{\pi}$; $policy_stable \leftarrow true$

Drawback:

Use policy evaluation in each step

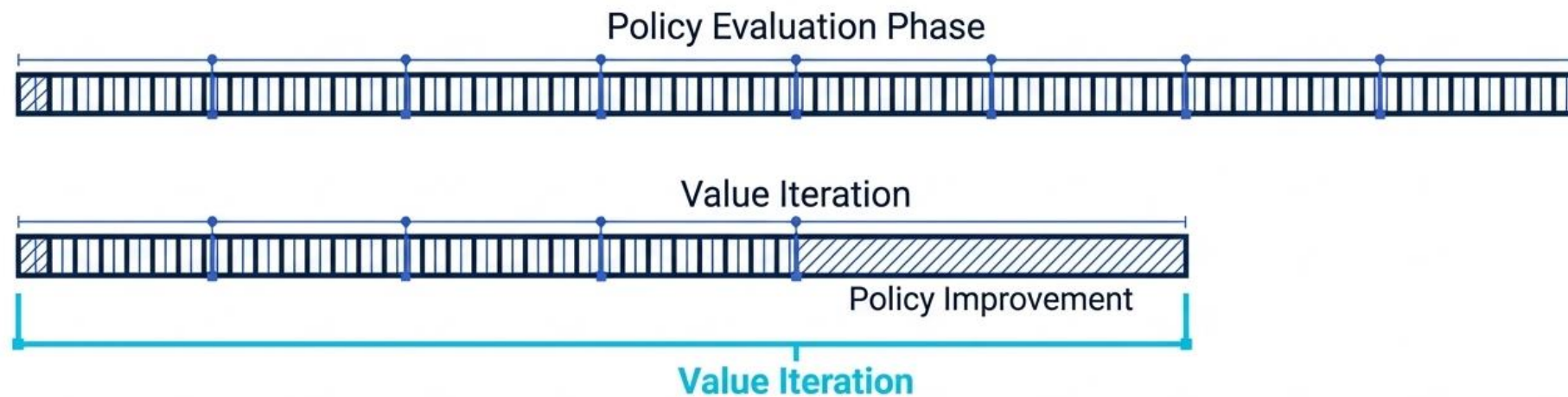
--> Computational and time consuming

- The term generalized policy iteration refers to all the ways we can interleave policy evaluation and policy improvement.
- We can relax it:
 - Each evaluation step brings our estimate a little closer to the value of the current policy but not all the way.
 - Each policy improvement step makes our policy a little more greedy, but not totally greedy.
- Intuitively, this process should still make progress towards the optimal policy and value function.



Value Iteration: Efficiency Through Truncation

Merging policy improvement and truncated evaluation into a single sweep.



The Bottleneck:

Waiting for Iterative Policy Evaluation to fully converge before making a policy improvement is computationally expensive.

The Truncation:

We can stop policy evaluation early, after just one single sweep of the state space.

The Update Rule:

By taking the maximum over all actions during the expected update, we effectively combine evaluation and improvement into a single step.

- Policy iteration fully evaluates a policy before improving it.
- Value iteration improves values immediately at every step.

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
```

until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

Aspect	Policy Evaluation	Policy Iteration	Value Iteration
Goal	Evaluate a fixed policy π .	Find the optimal policy π^* .	Find the optimal value function V^* .
Process	Computes $V\pi(s)$ iteratively.	Alternates between policy evaluation and improvement.	Iteratively updates $V(s)$ using the Bellman optimality equation.
Policy Update	No updates to the policy.	Updates policy in the improvement step.	Policy is derived after $V(s)$ converges.
Use Case	Analyze the performance of a specific policy.	Iteratively refine and improve a policy.	Efficiently compute both V^* and π^* .
Convergence Speed	Depends on policy evaluation only.	Typically slower than value iteration.	Often faster due to single-step updates.

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

- It is **highly effective for solving MDPs** when we **know** the transition probabilities and reward functions.
- It **guarantees convergence** with a complete and accurate model
- It requires a **complete model** of the environment, with **known transition probabilities** and rewards for each state-action pair
- DP algorithms, like Policy Iteration and Value Iteration, often **involve updating the value of every state in each iteration**, which can become **computationally intensive for large state spaces**
- DP methods are **planning** algorithms, meaning they **rely on a fixed model** and **can't naturally handle exploration** of unknown environments

Next

Model-free RL approaches (sample-based methods)

- Dozens of distinct algorithms exist beneath the math.
- They all answer just two independent questions.

Question 2: The Information

Question 1:

- What is the Goal ?

Question 2:

- What information do we have ?

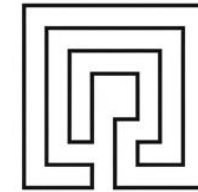


Planning

Use a known model.

Transition probabilities and rewards are known in advance.

Zero interaction required.



Learning

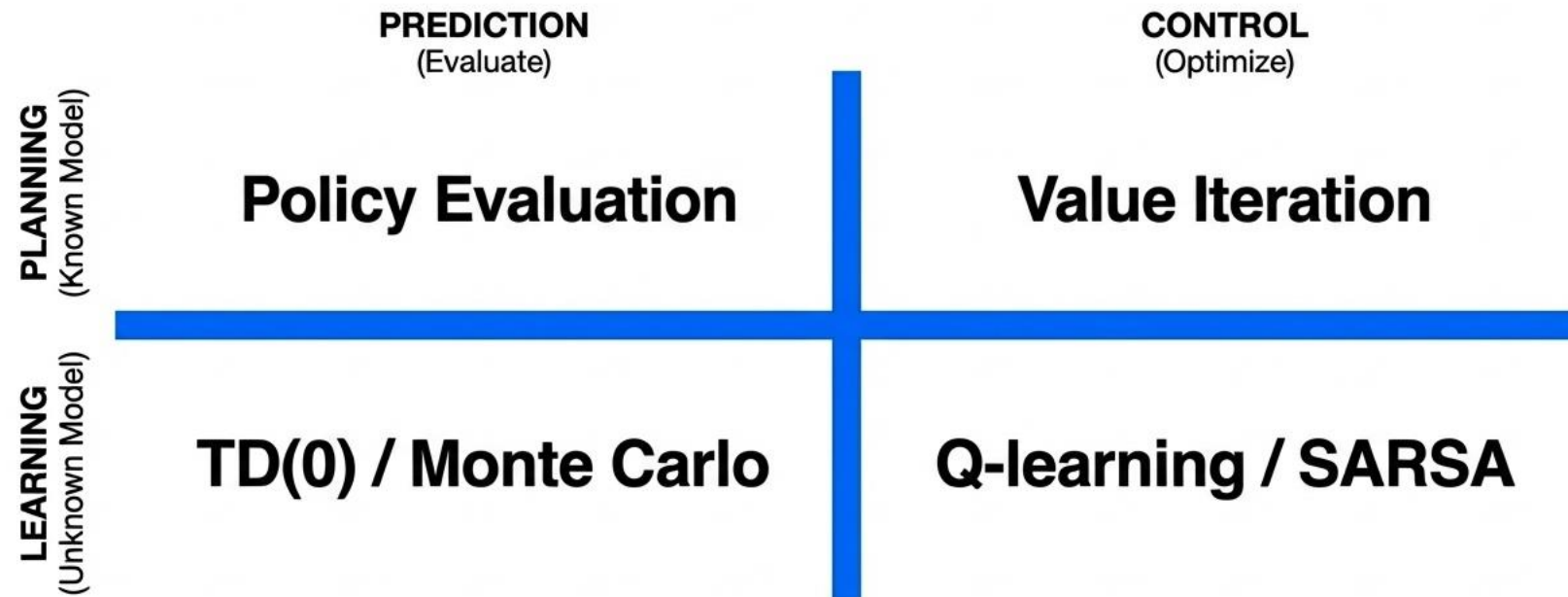
Learn from experience.

The model is completely unknown.

Must interact via samples (state, action, reward, next state).

The RL Matrix

Solving the same problem
under different constraints.



Prediction vs. Control is WHAT we solve. Planning vs. Learning is HOW we solve it.

Bellman Equations: Planning computes expectations exactly; Learning estimates them from samples.

- [1] Sutton, R. S. and Barto, A. G. (2018), Reinforcement Learning: An Introduction (Second edition). MIT Press
- [2] Adam White and Martha White, Reinforcement Learning Specialization, University of Alberta, Coursera